Beyond the Average Treatment Effect

# Estimators

MODIFIED
April 28, 2025

WEBR STATUS
Ready!

To recap, we have

1. defined a causal parameter $\theta$ that is represents the outcome $Y$ under a general hypothetical intervention, $\mathsf{d}(a_t, h_t, \epsilon)$, and
2. defined the necessary assumptions to identify the expected value of $Y$ under an intervention that replaces $A$ with the output of $\mathsf{d}_t(a, h_t, \epsilon)$.

The parameter $\theta$ is a function that could be computed if we knew the distribution of the variables (e.g., outcome regressions). Because we do not know this distribution, we need to estimate it from a sample of observed data. We now discuss how to estimate the parameter $\theta$ from a sample.

## Sequential Regression Estimator

One possible estimator is simply a *plug-in* estimator of the identification result. This estimator is often referred to as G-computation, and it proceeds by simply estimating the regressions described in the identification result. Another way to describe this algorithm is as follows:

1. Set $\mathsf{m}_{i,\tau+1} = Y_i$.

   For $t = \tau, \ldots, 1$:

   a. Using a pre-specified parametric model, regress $\mathsf{m}_{i,t+1}$ on $\{A_{i,t}, H_{i,t}\}$.

   b. Generate predictions from this model with $A_{i,t}$ changed to $A^{\mathsf{d}}_{i,t}$. Set $\mathsf{m}_{i,t}$ to be these predicted values.

   c. Repeat (iterate) the above two steps until setting $\mathsf{m}_{i,1}$ to the predicted values.

2. Take the final estimate as $\hat{\theta} = \frac{1}{n} \sum_{i=1}^{n} \hat{\mathsf{m}}_{i,1}$.

3. Compute standard errors using a bootstrap of steps 1 and 2.

A substitution estimator is nice, because its estimates are guaranteed to stay within the valid range of the outcome and it is simple to estimate. That said, its cons are major:

- In studies with multiple time points, the adjustment set can become large very quickly. For instance, consider a study with 3 covariates measured at every time point, and 10 time points. Even though we have only 3 covariates at each time point, the number of coviariates in the regression of $Y$ is 33.
- Imagine trying to correctly specify (e.g., include the approproate interactions) a parametric model (e.g., logistic, Cox) with 33 variables.

| **Pros** ✅ | **Cons** ❌ |
|---|---|
| Simple to implement | Requires correct estimation of all regressions |
| Substitution estimator | Requires pre-specified parametric models |

### Density-ratio estimation

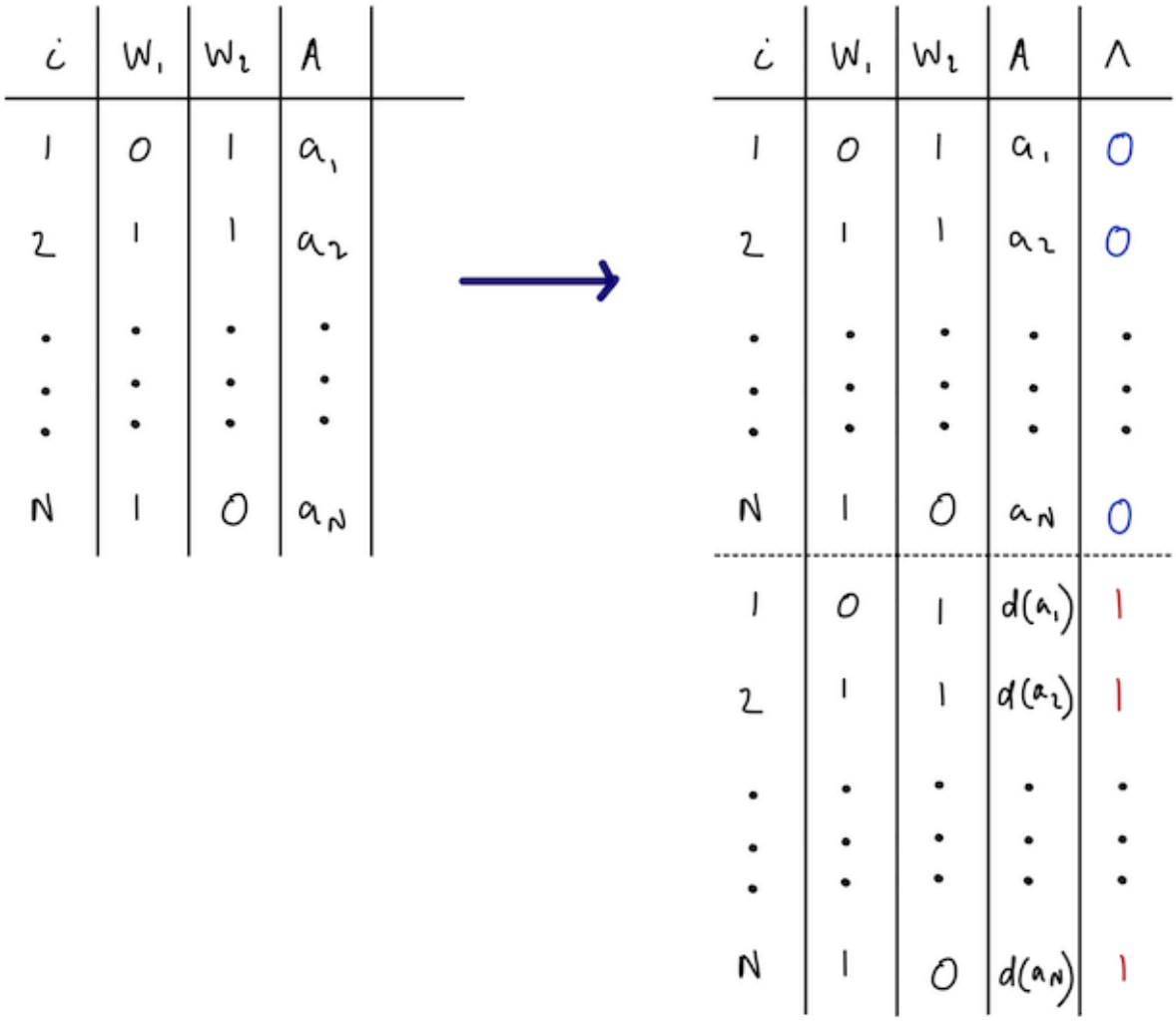
> The next three estimators all rely on estimating the density ratio
>
> $$r_t(a_t, h_t) = \frac{\mathrm{g}_t^{\mathsf{d}}(a_t \mid h_t)}{\mathrm{g}_t(a_t \mid h_t)}.$$
>
> Recall that $\mathrm{g}_t^{\mathsf{d}}(a_t \mid h_t)$ denotes the density of treatment *post-intervention* and that $\mathrm{g}_t(a_t \mid h_t)$ denotes the density of the observed treatment under no intervention.
>
> We will often refer to this ratio as the *intervention mechanism* throughout the workshop. Estimation of $r_t(a_t, h_t)$ is fully automated in `lmtp` and hidden from the user. **A comprehensive understanding of this process isn't necessary to use `lmtp`!**

- We can directly estimate this density ratio with a classification trick.
- Can compute the density ratio (which we can see is also the odds) in a classification problem. We use an augmented dataset with $2n$ observations. In this new dataset, the outcome is a new variable that we make, $\Lambda$, (defined below) and the predictors are the variables $A_t$ and $H_t$. In this new dataset, the data structure at time $t$ is redefined as

$$(H_{\lambda,i,t}, A_{\lambda,i,t}, \Lambda_{\lambda,i} : \lambda = 0, 1; i = 1, \ldots, n)$$

- $\Lambda_{\lambda,i} = \lambda_i$ indexes duplicate values. So if $\Lambda_i = 1$ if observation $i$ is a duplicated value and $\Lambda_i = 0$ otherwise.
  - For all duplicated observations $\lambda \in \{0, 1\}$ with the same $i$, $H_{\lambda,i,t}$ is the same
  - For all the non-duplicated observations, $\lambda = 0$, $A_{\lambda=0,i,t}$ equals the observed exposure values $A_{i,t}$

- For all the duplicated observations, $\lambda = 1$, $A_{\lambda=1,i,t}$ equals the exposure values under the intervention d , $A^{\mathsf{d}}_{i,t}$




Example of augmenting data for density ratio estimation.

- We then estimate the conditional probability that $\Lambda = 1$ in this dataset, and divide it by the corresponding estimate of the conditional probability that $\Lambda = 0$. Specifically, denoting $p^\lambda$ to be the distribution of the data in the augmented dataset, we have:

$$r_t(a_t, h_t) = \frac{p^\lambda(a_t, h_t \mid \Lambda = 1)}{p^\lambda(a_t, h_t \mid \Lambda = 0)} = \frac{p^\lambda(\Lambda = 1 \mid A_t = a_t, H_t = h_t)}{p^\lambda(\Lambda = 0 \mid A_t = a_t, H_t = h_t)}$$

**Example**

Consider the following example where we simulate 500 observations from this data-generating mechanism:

$$\begin{aligned}
L_1 &\sim \text{Normal}(0, 1) \\
A_1 &\sim \text{Bernoulli}(\text{logit}^{-1}(-1 + 1.5L_1)) \\
L_2 &\sim \text{Normal}(0, 1) \\
A_2 &\sim \text{Bernoulli}(\text{logit}^{-1}(-1 + 1.5L_2 + 2A_1)) \\
Y &\sim \text{Normal}(-1 - 1.2L_1 + 2.4A_1 - 2L_2 + 1.2A_2)
\end{aligned}$$

The data has been loaded into R in the background as `foo`. We will use the density ratio classification trick to estimate the density ratio $r_t(a_t, h_t)$ under the static intervention $d(a_t, h_t) = 1$. The true value under this intervention is $\approx 2.6$. First, we create the augmented data. We then regress the $\Lambda$ on $A_t$ and $H_t$ and estimate the density ratio as the predicted odds.

Run Code

```
# Create an empty matrix to store the density ratios
density_ratios <- matrix(NA, nrow = n, ncol = 2)

for (t in 1:2) {
  a <- c("A1", "A2")[t]
  # Duplicate the data
  shifted <- foo
  # In the duplicated data, modify treatment according to d
  shifted[[a]] <- 1

  # Set the values of delta in the original and modified data
  foo$lambda <- 0
  shifted$lambda <- 1

  # Stack the original data and the modified data
  density_ratio_data <- rbind(foo, shifted)

  # Regress lambda_t on A_t,H_t
  parents <- unlist(lapply(1:t, \(x) paste0(c("L", "A"), x)))
  formula <- as.formula(paste("lambda ~", paste(parents, collapse = "+")))
  density_ratio_model <- glm(formula, data = density_ratio_data, family = binomia

  # Calculate the estimated odds
  prob_lambda_1 <- predict(density_ratio_model, foo, type = "response")
  density_ratios[, t] <- prob_lambda_1 / (1 - prob_lambda_1)
}

summary(density_ratios)
```

## Inverse Probability Weighting

We call this estimator IPW due to its similarity with the inverse-probability weighted estimator for binary treatments, but it would be more accurately referred to as simply reweighted estimator. It is based on the following alternative identification formula:

$$\theta = \mathsf{E}\left[\left\{\prod_{t=1}^{\tau} r_t(a_t, h_t)\right\} Y\right]$$

The algorithm is as follows:

> 1. Construct estimates of $r_{i,t}(a_t, h_t)$ using the density ratio classification trick and a pre-specified parametric model.
> 2. Define the weights $w_i = \prod_{t=1}^{\tau} r_{i,t}(a_t, h_t)$.
> 3. Take the final estimate as $\hat{\theta} = \frac{1}{n} \sum_{i=1}^{n} \hat{w}_i \times y_i$.
> 4. Compute standard errors using a bootstrap of steps 1-3.

**Example**

Let's apply the IPW estimator using the estimated density ratios we calculated in the previous example. We first compute the weights as the product of the density ratios.

Run Code

```
weights <- apply(density_ratios, 1, prod)
```

Our estimate is then the sample average of the estimated weights times the observed outcomes.

Run Code

```
mean(weights*foo$Y)
```

Similarly to the sequential regression estimator, the IPW is simple to implement. However, it suffers from the same drawbacks.

| Pros ✅ | Cons ❌ |
|---|---|
| Simple to implement | Requires correct estimation of all regressions |
| | Requires pre-specified parametric models |

## Doubly Robust Estimators

G-computation and IPW estimators require the estimation of nuisance parameters with correctly specified parametric models. We will now turn our attention to two non-parametric estimators:

1. targeted minimum-loss based estimator (TMLE), and
2. a sequentially doubly-robust estimator (SDR).

Wait, what does it mean for an estimator to be doubly robust?

- For the simple case of a single time point, an estimator is considered doubly robust if it is able to produce a consistent estimate of the target parameter as long as either the outcome model is consistently estimated or the treatment (and censoring) model(s) are consistently estimated. For example:

| | |
|---|---|
| Time | 1 |
| Treatment | Correct |
| Outcome | Wrong |

- For time-varying setting, an estimator is doubly robust if, for some time $s$, all outcome regressions for $t > s$ are consistently estimated and all intervention mechanisms (treatment + censoring) for $t \leq s$ are consistently estimated. Consider for example $5$ time points:

| Time | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Treatment | Correct | Correct | Wrong | Wrong | Wrong |
| Outcome | Wrong | Wrong | Correct | Correct | Correct |

- Sequential double robustness (often also referred to as $2^{\tau}$-multiply robust) implies that an estimator is consistent if for all times either the outcome or intervention mechanism (treatment + censoring) is consistently estimated. For example:

| Time | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Treatment | Correct | Wrong | Correct | Wrong | Correct |
| Outcome | Wrong | Correct | Wrong | Correct | Wrong |

> Be careful not to confuse doubly robust consistency with *rate double robustness*. Doubly robust consistency refers to the properties we just described. Rate double robustness refers to the property where the error of an estimator is the product of nuisance functions.

### Efficient Influence Function

Key to constructing the TMLE and SDR is the *efficient influence function* (EIF).

> - The EIF characterizes the asymptotic behavior of all regular and efficient estimators.
> - The EIF characterizes the first-order bias of pathwise differentiable estimands.

Before we introduce the EIF, it's necessary to make some additional assumptions on $A$ and $\mathsf{d}$.

1. The treatment $A$ is discrete, or
2. If $A$ is continuous, the function $\mathsf{d}$ is piecewise smooth invertible
3. The function $\mathsf{d}$ does not depend on the observed distribution $\mathsf{P}$ (this means that we cannot apply these methods to odds ratio IPSI, although we can apply them to risk ratio IPSI)

These assumptions ensure that the efficient influence function of $\theta$ for interventions $\mathsf{d}$ have a structure similar to the influence function for the effect of dynamic regimes. This allows for multiply robust estimation, which is not generally possible for interventions $\mathsf{d}$ that depend on $\mathsf{P}$.

Define the function

$$\phi_t : O \mapsto \sum_{s=t}^{\tau} \left( \prod_{k=t}^{s} r_k(a_k, h_k) \right) \{ \mathsf{m}_{s+1}(a_{s+1}^{\mathsf{d}}, h_{s+1}) - \mathsf{m}_s(a_s, h_s) \} + \mathsf{m}_t(a_t^{\mathsf{d}}, h_t).$$

The efficient influence function for estimating $\theta = \mathsf{E}[\mathsf{m}_1(A^{\mathsf{d}}, L_1)]$ in the non-parametric model is given by $\phi_1(O) - \theta$.

In the case of single time-point, the influence function simplifies to

$$r(a, w)\{Y - \mathsf{m}(a, w)\} + \mathsf{m}(a^{\mathsf{d}}, w) - \theta.$$

### Targeted Minimum-Loss Based Estimation

| Pros ✅ | Cons ❌ |
|---|---|
| Substitution estimator | not sequentially doubly-robust |
| doubly-robust | |
| can use machine learning | |

The algorithm is as follows:

> 1. Construct estimates of $r_{i,t}(a_t, h_t)$ using the density ratio classification trick and your favorite regression method.
> 2. For $t = 1, \ldots, \tau$, compute the weights: $w_{i,t} = \prod_{k=1}^{t} r_{i,k}(a_{i,k}, h_{i,k})$
> 3. Set $\tilde{\mathsf{m}}_{i,\tau+1}(A_{i,t+1}^{\mathsf{d}}, H_{i,t+1}) = Y_i$.
>
>  For $t = \tau, \ldots, 1$:
>
>  1. Regress $\tilde{\mathsf{m}}_{i,t+1}(A_{i,t+1}^{\mathsf{d}}, H_{i,t+1})$ on $\{A_{i,t}, H_{i,t}\}$.

- Using this regression, generate predictions from $\{A_{i,t}, H_{i,t}\}$ and $\{A^{\mathsf{d}}_{i,t}, H_{i,t}\}$.
- Denote the predictions as $\tilde{\mathsf{m}}_t(A_{i,t}, H_{i,t})$ and $\tilde{\mathsf{m}}_t(A^{\mathsf{d}}_{i,t}, H_{i,t})$ respectively.

2. Fit the generalized linear tilting model:

$$\text{link } \tilde{\mathsf{m}}^{\epsilon}_t(A_{i,t}, H_{i,t}) = \epsilon + \text{link } \tilde{\mathsf{m}}_{i,t}(A_{i,t}, H_{i,t})$$

with weights $w_{i,t}$.

- $\text{link } \tilde{\mathsf{m}}_{i,t}(A_{i,t}, H_{i,t})$ is an offset variable (i.e., a variable with known parameter value equal to one).
- The parameter $\epsilon$ may be estimated by running a generalized linear model of $\tilde{\mathsf{m}}_{i,t+1}(A^{\mathsf{d}}_{i,t+1}, H_{i,t+1})$ with only an intercept term, an offset term equal to $\text{link } \tilde{\mathsf{m}}_{i,t}(A_{i,t}, H_{i,t})$, and weights $w_{i,t}$.

3. Let $\hat{\epsilon}$ be the maximum likelihood estimate, and update the estimates as:

$$\text{link } \tilde{\mathsf{m}}^{\hat{\epsilon}}_t(A^{\mathsf{d}}_{i,t}, H_{i,t}) = \hat{\epsilon} + \text{link } \tilde{\mathsf{m}}_t(A^{\mathsf{d}}_{i,t}, H_{i,t})$$

$$\text{link } \tilde{\mathsf{m}}^{\hat{\epsilon}}_t(A_{i,t}, H_{i,t}) = \hat{\epsilon} + \text{link } \tilde{\mathsf{m}}_t(A_{i,t}, H_{i,t})$$

4. Update $\tilde{\mathsf{m}}_{i,t} = \tilde{\mathsf{m}}^{\hat{\epsilon}}_{i,t}$, $t = t - 1$, and iterate.

4. The final estimate is defined as $\hat{\theta} = \frac{1}{n}\sum_{i=1}^{n} \tilde{m}_{i,1}(A^{\mathsf{d}}_{i,1}, L_{i,1})$.

**Example**

Let's apply TMLE using the estimated density ratios we calculated in the previous example.

Run Code

```
# First compute the weights as the cumulative product of the density ratios
weights <- t(apply(density_ratios, 1, cumprod))

predictions_natural <- matrix(NA, nrow = n, ncol = 2)
predictions_shifted <- matrix(NA, nrow = n, ncol = 2)

# Set t = 2
# Because t + 1 = tau + 1, set the first pseudo outcome to the observed outcome
m3_d <- foo$Y

# Regress the pseudo outcome on the observed data
fit2 <- glm(m3_d ~ A2 + L2 + A1 + L1, data = foo)

# Generate the predictions for t = 2
predictions_natural[, 2] <- predict(fit2)
predictions_shifted[, 2] <- predict(fit2, mutate(foo, A2 = 1))

# Perform the targeting step
targeting_fit2 <- glm(m3_d ~ offset(predictions_natural[, 2]), weights = weights[
```

```

# Update predictions
predictions_natural[, 2] <- predictions_natural[, 2] + coef(targeting_fit2)
predictions_shifted[, 2] <- predictions_shifted[, 2] + coef(targeting_fit2)

# Iterate, setting t - 1 = 1
# The new pseudo outcome is the targeted outcome under the shift at t + 1 = 2
m2_d <- predictions_shifted[, 2]

# Regress the pseudo outcome on the observed data
fit1 <- glm(m2_d ~ A1 + L1, data = foo)

# Generate the predictions for t = 2
predictions_natural[, 1] <- predict(fit1)
predictions_shifted[, 1] <- predict(fit1, mutate(foo, A1 = 1))

# Perform the targeting step
targeting_fit1 <- glm(m2_d ~ offset(predictions_natural[, 1]), weights = weights[

# Update predictions
predictions_natural[, 1] <- predictions_natural[, 1] + coef(targeting_fit1)
predictions_shifted[, 1] <- predictions_shifted[, 1] + coef(targeting_fit1)

mean(predictions_shifted[, 1])
```

### Sequentially Doubly Robust Estimator

| Pros ✅ | Cons ❌ |
|---|---|
| doubly-robust | not a substitution estimator |
| sequentially doubly-robust | |
| can use machine learning | |

The algorithm is as follows:

> 1. Construct estimates of $r_{i,t}(a_t, h_t)$ using the density ratio classification trick and your favorite regression method.
>
> 2. Initialize $\phi_{\tau+1}(O_i) = Y_i$.
>
>  For $t = \tau, \ldots, 1$:
>
>  1. Compute the pseudo-outcome $\check{Y}_{i,t+1} = \phi_{t+1}(O_i)$.
>
>  2. Regress $\check{Y}_{i,t+1}$ on $\{A_{i,t}, H_{i,t}\}$.

3. Let $\check{\mathbf{m}}_{i,t}$ denote the predicted values. Update $\mathbf{m}_{i,t} = \check{\mathbf{m}}_{i,t}$ and iterate.

3. The final estimate is defined as $\hat{\theta} = \frac{1}{n}\sum_{i=1}^{n} \phi_1(O_i)$.

**Example**

Let's apply the SDR estimator. Again, we'll use the estimated density ratios we previously estimated.

Run Code

```
predictions_natural <- matrix(NA, nrow = n, ncol = 2)
predictions_shifted <- matrix(NA, nrow = n, ncol = 3)

# Set t = 2
# Because t + 1 = tau + 1, set the first pseudo outcome to the observed outcome
m3_d <- foo$Y
predictions_shifted[, 3] <- m3_d

# Regress the pseudo outcome on the observed data
fit2 <- glm(m3_d ~ A2 + L2 + A1 + L1, data = foo)

# Generate the predictions for t = 2
predictions_natural[, 2] <- predict(fit2)
predictions_shifted[, 2] <- predict(fit2, mutate(foo, A2 = 1))

# Iterate, setting t - 1 = 1
# Compute the pseudo outcome using the efficient influence function
m2_d <- density_ratios[, 2]*(m3_d - predictions_natural[, 2]) + predictions_shift

# Regress the pseudo outcome on the observed data
fit1 <- glm(m2_d ~ A1 + L1, data = foo)

# Generate the predictions for t = 1
predictions_natural[, 1] <- predict(fit1)
predictions_shifted[, 1] <- predict(fit1, mutate(foo, A1 = 1))

weights <- t(apply(density_ratios, 1, cumprod))

# Compute the estimate as the sample mean of the uncentered efficient influence f
uc_eif <- rowSums(weights * (predictions_shifted[, 2:3] - predictions_natural[, 1
  predictions_shifted[, 1]

mean(uc_eif)
```

## Choosing an Estimator

- In general we never recommend using the IPW or sequential regression estimator. Both require the use of correctly pre-specified parametric models for valid statistical inference 😳.
- The TMLE and SDR estimators, however, are both doubly or sequentially doubly robust and can be used with machine-learning algorithms while remaining $\sqrt{n}$-consistent under reasonable assumptions.

**Table 1.** Summary of estimator properties.

| | IPW | G-comp. | TMLE | SDR |
|---|---|---|---|---|
| Uses outcome regression | | ⭐ | ⭐ | ⭐ |
| Uses the propensity score | ⭐ | | ⭐ | ⭐ |
| Valid inference with machine-learning | | | ⭐ | ⭐ |
| Substitution estimator | | ⭐ | ⭐ | |
| Doubly robust | | | ⭐ | ⭐ |
| Sequentially doubly robust | | | | ⭐ |

**Question**: Why can't we use machine learning with the G-computation and IPW estimators?

▶ ✅ Answer

While the SDR estimator may be more robust to model misspecification, the TMLE does have the advantage of being a substitution estimator. Because of this, estimates from the TMLE are guaranteed to stay within the valid range of the outcome. Taken together, this leads to the following recommendations for choosing between the TMLE and SDR:

- If treatment is not time-varying, use the TMLE.
- If treatment is time-varying and the parameter $\theta$ has clear bounds, such as probabilities, beware of the SDR estimator. Use TMLE preferably.

## Cross-fitting

When estimating nuisance parameters with data adaptive algorithms, you should perform a process similar to cross-validation called cross-fitting. Cross-fitting helps ensure:

- that standard errors will be correct, and
- can help reduce estimator bias and improve coverage of the confidence intervals.

Cross-fitting is fully automated in `lmtp`, but for more information we recommend reviewing Chernozhukov et al. (2018), Díaz (2020), and Zivich and Breskin (2021).

---

## References


Bickel, Peter J, Chris AJ Klaassen, Peter J Bickel, Ya'acov Ritov, J Klaassen, Jon A Wellner, and YA'Acov Ritov. 1993. *Efficient and Adaptive Estimation for Semiparametric Models*. Vol. 4. Springer.

Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. 2018. "Double/Debiased Machine Learning for Treatment and Structural Parameters." Oxford University Press Oxford, UK.

Díaz, Iván. 2020. "Machine Learning in the Estimation of Causal Effects: Targeted Minimum Loss-Based Estimation and Double/Debiased Machine Learning." *Biostatistics* 21 (2): 353–58.

Díaz, Iván, Nicholas Williams, Katherine L Hoffman, and Edward J Schenck. 2023. "Nonparametric Causal Effects Based on Longitudinal Modified Treatment Policies." *Journal of the American Statistical Association* 118 (542): 846–57.

Haneuse, Sebastian, and Andrea Rotnitzky. 2013. "Estimation of the Effect of Interventions That Modify the Received Treatment." *Statistics in Medicine* 32 (30): 5260–77.

Hoffman, Katherine L., Diego Salazar-Barreto, Nicholas Williams, Kara E. Rudolph, and Ivan Diaz. 2024. "Studying Continuous, Time-Varying, and/or Complex Exposures Using Longitudinal Modified Treatment Policies." https://arxiv.org/abs/2304.09460.

Kennedy, Edward H. 2019. "Nonparametric Causal Effects Based on Incremental Propensity Score Interventions." *Journal of the American Statistical Association* 114 (526): 645–56.

Zivich, Paul N, and Alexander Breskin. 2021. "Machine Learning for Causal Inference: On the Use of Cross-Fit Estimators." *Epidemiology* 32 (3): 393–401.